

Oficina de Programação de Jogos CAJUFPR na SACI



Programação de Hoje

⇒ Apresentação Rápida do Godot

Instalação & Passeando pela Engine Gráfica

⇒ Conceitos da Linguagem GDScript

Variáveis, Listas, Objetos, e tudo que há de bom

⇒ Programando Funcionalidades

Funções da Engine + Conceitos básicos do jogo

⇒ Mão na Massa!

Implementação de uma biblioteca de um Top-Down Shooter!



Godot

⇒ O Godot é uma Engine Gráfica para jogos lançada oficialmente em 2014. Seu código é *open source*, além de ser leve, eficiente e nativamente executar em Linux.

⇒ Assim como a maioria das Game Engines, é focada fortemente na Programação Orientada a Objetos, com os elementos do jogo sendo **nodos** e **cenas**.

⇒ É possível alterar completamente os elementos do jogo via código, mas o Godot também fornece uma boa integração **visual** com as variáveis.

Vamos instalar o Godot e passear pela Game Engine!



GScript

- ⇒ Linguagem usada nos códigos de programação dos elementos dentro do Godot.
- ⇒ Como comentado anteriormente, temos os **nodos** que são os objetos do código, as **cenas** que são conjuntos de nodos estruturados em árvore, e os **scripts** que são códigos ligados a nodos e cenas, que permitem **mudar o comportamento deles**.
- ⇒ Temos dois tipos predominantes dentro da Engine:
 - **Arquivos .tscn**: São as cenas do jogo. Elas servem principalmente para modularizar os objetos do jogo.
 - **Arquivos .gd**: São os scripts do jogo. É possível criar e conectar um script a um nodo pela própria Engine.

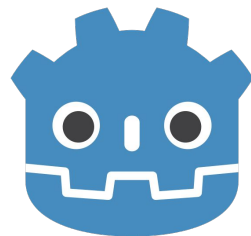


GScript

⇒ Sobre a linguagem em si, ela é **bem parecida com Python**, então ela é **baseada em indentação**, além de possuir **tipagem dinâmica**.

⇒ Temos os seguintes elementos da programação:

- **Variáveis:** Inicie uma variável usando `var <nome>`.
- **Funções:** Inicie uma função através de `func <nome>():` e logo em seguida o corpo da função.
- **Listas e Tuplas:** Vetores com tipos variados são as listas e tuplas, e são usadas através de `[a, b, c]` e `(x, y, z)`.
- **Condicionais e Loops:** Como em Python e outras linguagens, usa-se `if <condição>:` e `else:` e também `for x in range(...):` e `while <condição>:.`



GODOT
Game engine

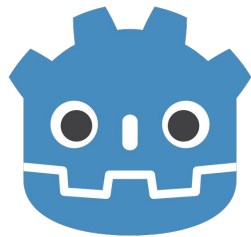


Signals

⇒ Como estamos trabalhando com **objetos**, temos que fazer a comunicação entre eles de alguma maneira. O jeito que o Godot faz é através do padrão **Observer**.

⇒ Quando um objeto realiza uma ação e quer notificar outros objetos que algo foi feito, ele **emite um sinal**. Quando outros objetos, **conectados nesse sinal**, recebem essa notificação, eles também farão algo.

⇒ **Exemplo:** Seu jogador disparou uma bala em um inimigo, que o acertou. A bala então deve emitir um sinal `hit_bullet` que notifica o objeto do inimigo que ele foi acertado, e assim decrementar sua própria vida.



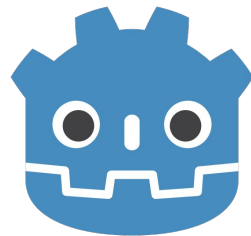
GODOT
Game engine



Signals

- ⇒ Para **criar um sinal** dentro do código, basta colocar no topo dizendo `signal <nome_do_sinal>`.
- ⇒ Para **emitir um sinal** dentro do código, podemos que chamar a função própria do Godot `emit_signal(<nome_do_sinal>)` ou de forma mais moderna `<nome_do_sinal>.emit()`.
- ⇒ Para **conectar a um sinal** dentro do código, temos que chamar o método de conexão em cima do nome do sinal, ou seja, `<nome_do_sinal>.connect(<função_de_sinal>)`. Mas também temos que ter uma função **que faz algo quando o sinal é ativado**, com o nome padronizado:

```
func <função_de_sinal>():, onde <função_de_sinal> é  
    _on_<objeto-emissor>_<nome_do_sinal>
```



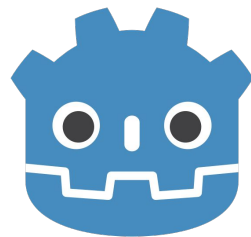
GODOT
Game engine



Funções e Objetos do Godot

⇒ O Godot possui funções próprias da Engine para realizar certos procedimentos:

- **Função `_init()`**: Tudo que estiver dentro desta função é executada **no momento que o nodo é criado na memória**. Coloque tudo que o nodo precisa quando surgir.
- **Função `_ready()`**: Tudo que estiver dentro desta função é executada **no momento que o nodo aparece na tela**. Coloque tudo que o nodo precisa quando aparecer.
- **Função `_process(delta)`**: Tudo que estiver dentro dessa função é executada **a cada frame**. Ela recebe um parâmetro especial chamado **delta**, que informa o tempo entre um frame e outro. Isso é importante, pois esse valor não é fixo (60 frames por segundo não significa que o intervalo entre eles é igual).



GODOT
Game engine



Funções e Objetos do Godot

⇒ Além disso, existem objetos padrão da Engine para diversos procedimentos:

- **Objeto `Vector2`**: Representa um vetor de duas dimensões dentro do Godot. Ele é imprescindível para cálculo vetorial, como posição do jogador, velocidade, trajetória, etc. Possui métodos próprios por conta de ser um objeto.
- **Objeto `Input`**: Serve para receber entradas do jogador. Apresenta métodos tanto para ler uma tecla como entender qual movimento está sendo recebido, e consegue retornar um `Vector2` como resposta.



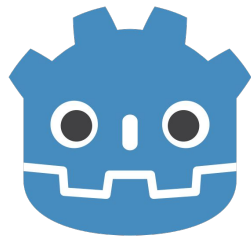
Mão na Massa!

⇒ Agora é hora de implementar três funcionalidades dentro de um jogo! Acessem esse link do Github e clonem o repositório, ou digitem o seguinte comando:

```
git clone https://github.com/sergingameprays/dodge_creeps_mods
```

Implementem as seguintes funções no script **utils.gd**:

- **Função `use_potion()`**: Se a poção não foi usada ainda, utilize ela e cure o jogador.
- **Função `try_dash()`**: Verifica se é possível usar o dash do jogador e, se sim, realizar o movimento.
- **Função `apply_weapon()`**: Aplica a arma ao jogador, mudando sua arma através da função de configuração.
- **Função `switch_weapon()`**: Decide qual é a arma e aplica ela.



GODOT
Game engine

